



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA



PROJEKAT IZ PREDMETA
NAPREDNI MIKROPROCESORSKI SISTEMI

**Implementacija RISC-V procesora sa
protočnom obradom i podsistemom keš
memorije za podatke**

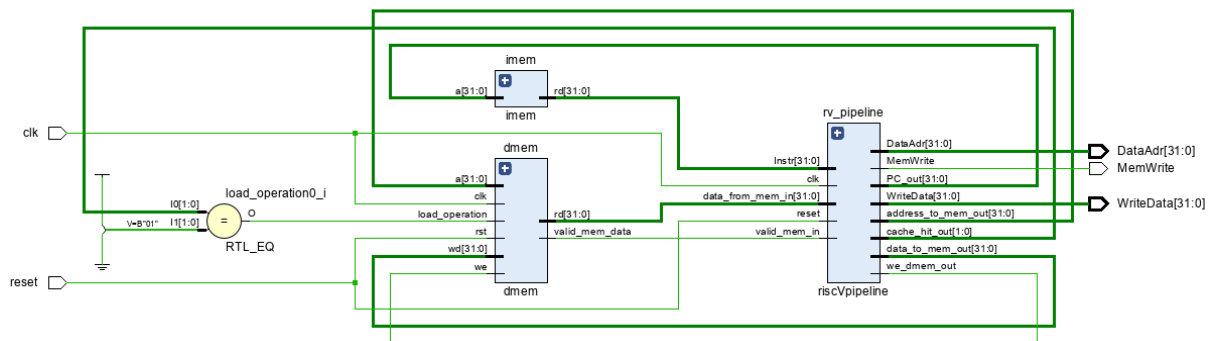
Asistent : Nebojša Pilipović

Student : Aleksandar Vig

Specifikacija sistema

- RISC 32-bitni procesor sa protočnom obradom
- Dvosmerno set asocijativna keš memorija za podatke prvog nivoa
 - LRU polisa evikcije
 - Kapacitet memorije 64 seta sa dva smera
 - Write-back polisa upisa keš memorije
 - Bajt adresibilna memorija
- Četvorosmerno set asocijativna keš memorija za podatke drugog nivoa
 - Pseudo LRU (NMRU) polisa evikcije
 - Kapacitet memorije 64 seta sa četiri smera
 - Write-back polisa upisa keš memorije
 - Bajt adresibilna memorija
- Glavna memorija za podatke
 - Mogućnost upisa 2048 podataka širine 32 bita

Procesor



1. Komponente procesora

Na slici 1. prikazane su komponente od kojih je sačinjen procesor :

- Instrukciona memorija u kojoj su smeštene instrukcije koje je potrebno izvršiti.
- Procesorsko jezgro koje je zaduženo da izvršava instrukcije pročitane iz instrukcione memorije.
 - Između procesora i memorije za podatke unutar procesora implementiran je podsistem keš memorija. Sačinjen je od keš memorije za podatke prvog i drugog nivoa.
- Memorija za podatke

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

2. Format instrukcija

Za kodovanje instrukcija se koriste formati prikazani na slici 1. Instrukcije su podeljene na polja i u zavisnosti od funkcije koju oni imaju u instrukciji mogu biti prisutna polja:

- opcode - kod instrukcije, vrši osnovnu podelu instrukcija.
- funct3 - dodatna 3 bita polju opcode, za detaljnije definisanje instrukcije.
- funct7 - dodatnih 7 bita polju opcode, za detaljnije definisanje instrukcije.
- rs1 – adresa prvog operanda.
- rs2 – adresa drugog operanda.
- rd - ciljni registar, u koji se smešta rezultat instrukcije.
- imm - immediate, polje koje sadrži konstantu.

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0

3. Implementirane instrukcije R formata

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
addi	ADD Immediate	I	0010011	0x0		rd = rs1 + imm
xori	XOR Immediate	I	0010011	0x4		rd = rs1 ^ imm
ori	OR Immediate	I	0010011	0x6		rd = rs1 imm
andi	AND Immediate	I	0010011	0x7		rd = rs1 & imm
slli	Shift Left Logical Imm	I	0010011	0x1	imm[5:11]=0x00	rd = rs1 << imm[0:4]
srl	Shift Right Logical Imm	I	0010011	0x5	imm[5:11]=0x00	rd = rs1 >> imm[0:4]
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0

4. Implementirane instrukcije I formata

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
beq	Branch ==	B	1100011	0x0		if(rs1 == rs2) PC += imm

5. Implementirana instrukcija B formata

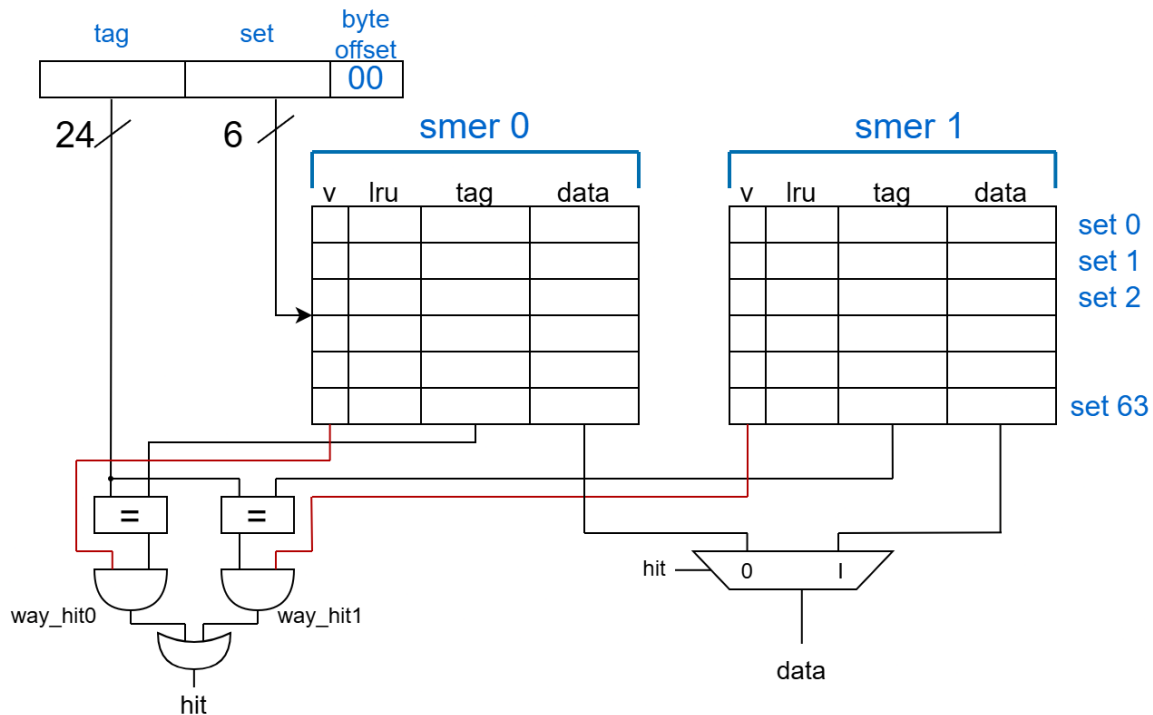
Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
jal	Jump And Link	J	1101111			rd = PC+4; PC += imm

6. Implementirana instrukcija J formata

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]

7. Implementirane instrukcije čitanja i upisa u memoriju

Keš memorija prvog nivoa za podatke



8. Logika po kojoj je implementirana L1 keš memorija

Na slici 7. prikazana je keš memorija prvog nivoa koja je dvosmerno set asocijativna i ima write-back polisu upisa u keš memoriju.

Posедуje 64 seta i svaki set ima dva smera, polja svakog smera su:

- Polje **data** - podatak 32-bitni.
- Polje **v** - kojim se označava da li je keširani podatak validan.
- Polje **lru** - kojim se označava koji od dva smera je davnije korišćen
- Polje **tag** - 24-bitna vrednost koja predstavlja ostatak adrese koji nije iskorišćen za indeksiranje podatka.

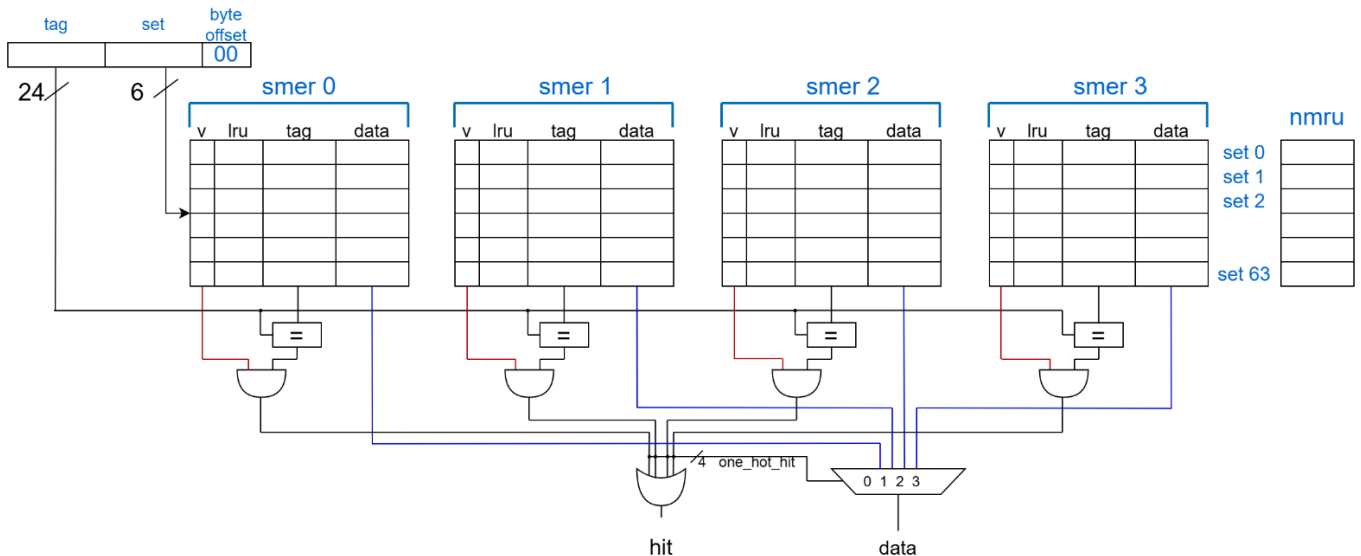
Logički gledano, keš memorija se može podeliti u više manjih celina :

- Kombinatorno-sekvencijalna logika kojom je implementirana mašina stanja. Poseduje *MAIN* i *WAIT_WRITE* stanje. Inicijalno je u stanju *MAIN*, pri operaciji čitanja proverava se da li je podatak prisutan u keš memoriji, ukoliko dođe do promašaja pri čitanju prelazi se u stanje *WAIT_WRITE* i u njemu ostaje sve dok se podatak ne učita iz keš

memorije drugog nivoa. Tokom boravka u stanju *WAIT_WRITE* na izlazni port *stall* postavljena je jedinica čime se zamrzava ostatak procesora kako se ne bi naredna instrukcija izvršila.

- Kombinatorna logika kojom se u slučaju operacije čitanja u slučaju pogotka traženi podatak postavlja na port *rd*.
- Kombinatorno-sekvencijalna logika kojom je implementiran upis podataka u jedan od smerova i ažuriranje *valid* i *lru* polja unutar svakog smera keš memorije. Implementirani su slučajevi :
 - Kada su oba smera pri operaciji upisa nevalidna, tada se podatak smešta na u prvi smer, ažurira se valid polje tog smera.
 - Kada je prvi smer validan, tada se u slučaju upisa prvo proverava da li se tag novog podatka poklapa sa tagom podatka koji je već prisutan u keš memoriji. Ukoliko je već prisutan tag, podatak se samo ažurira, u suprotnom podatak čiji se tag ne poklapa smešta se u drugi smer. Nakon toga oba smera postaju validna i lru polja su ažurirana.
 - Kada su oba smera validna, tada se pri operaciji upisa proverava da li je tag već prisutan u jednom od oba smera. Ukoliko je prisutan tag tada se podatak i lru stanje se samo ažurira. Ukoliko tag nije prisutan novopridošli podatak upisuje se u davnije korišćeno polje, a podatak i tag koju su se pre nalazili eviktuju se u keš memoriju drugog nivoa.
 - Pri operaciji čitanja kada dođe do promašaja prvo se proverava da li su oba smera nevalidna, to će se dogoditi pri obaveznom promašaju. Ako su oba smera nevalidna podatak se upisuje u prvi smer. U suprotnom podatak se upisuje u smer koji je davnije korišćen. Takođe ukoliko je prethodni podatak bio valid a tagovi se ne poklapaju taj podatak se eviktuje iz L1 u L2 keš memoriju.
 - Pri operaciji čitanja kada dođe do pogotka, tada se proverava u kom smeru se dogodio pogodak i ažurira se lru stanje oba smera.

Keš memorija drugog nivoa za podatke



9. Logika po kojoj je implementirana keš memorija drugog nivoa

Na slici 8. prikazana je keš memorija drugog nivoa koja je dvosmerno set asocijativna i ima write-back polisu upisa u keš memoriju.

Poseduje 64 seta i svaki set ima četiri smer, polja svakog smer :

- Polje **data** - podatak 32-bitni podaci.
- Polje **v** - kojim se označava da li je keširani podatak validan.
- Polje **tag** - 24-bitna vrednost koja predstavlja ostatak adrese koji nije iskorišćen za indeksiranje podatka.

```
case (mru[index])
  2'd0: target_way = 1;
  2'd1: target_way = 2;
  2'd2: target_way = 3;
  2'd3: target_way = 0;
endcase
```

Nezavisno od prethodno navedenih polja, keš memorija poseduje niz **nmru** od 64 2-bitna podatka kojim se vodi evidencija koji smer je poslednji korišćen unutar jednog seta. Kada je potrebno da se podatak eviktuje iz keš memorije ukoliko je poslednje korišćen smer 0 tada će se eviktovati podatak iz smer 1 i u taj smer će biti upisan novi podatak. Ukoliko je korišćen poslednje smer 1 biće eviktovan podatak iz smer 2. Slika iznad prikazuje logiku po kojoj je implementirana NMRU polisa evikcije.

Logički gledano, keš memorija se može podeliti u više manjih celina :

- Kombinatorna logika kojom se proverava da li je tag traženog podatka prisutan u keš memoriji. Iskorišćeno je 4 bita za promenljivu *hit_way* i obeležava se kao one-hot, samo jedan bit unutar promenljive može biti 1.
- Kombinatorno-sekvencijalna logika kojom je implementirana mašina stanja. Posедуje *MAIN* i *MISS* stanje. Inicijalno je u stanju *MAIN*, pri operaciji čitanja proverava se da li je podatak prisutan u keš memoriji, ukoliko dođe do promašaja pri čitanju prelazi se u stanje *MISS* i u njemu se ostaje sve dok se podatak ne učita iz glavne memorije.
- Kombinatorna logika kojom se u slučaju operacije čitanja i pogotka pri čitanju traženi podatak postavlja na port *rd*.
- Kombinatorna logika kojom se u slučaju pogotka ažurira vrednost nmr u stanja seta.
- Kombinatorno-sekvencijalna logika kojom je implementirana keš četvorosmerno set asocijativna memorija drugog nivoa. Logika se može podeliti u više manjih celina :
 - Prilikom operacije upisa proverava se da li je tag podatka već prisutan u jednom od smerova keš memorije. Ukoliko je prisutan tag, podatak se samo ažurira.
 - U slučaju promašaja pri upisu proverava se da li postoji smer u koji nikad do sad nije upisan podatak, tj proverava se da li postoji smer čiji podatak nije validan. Ukoliko postoji takav smer podatak se upisuje u taj smer i ažurira se valid bit tog smeru. Ukoliko su svi smerovi validni i tag se ne nalazi ni u jednom od smerova potrebno je eviktovati jedan od smerova u zavisnosti od NMRU vrednosti rezervirane za taj set. Prethodni podatak potrebno je eviktovati kako bi se u taj smer upisao podatak sa novopridošlim tagom.
 - Pri operaciji čitanja ukoliko dođe do pogotka pri čitanju NMRU vrednost za taj set se ažurira.
 - Pri operaciji čitanja ukoliko dođe do promašaja, prvo se proverava da li postoji smer čiji podatak nije validan i u taj smer se onda upisuje podatak. Ukoliko su svi smerovi validni, tada je potrebno eviktovati podatak iz smeru shodno vrednosti NMRU i u taj smer je potrebno upisati nov podatak.

Analiza rada sistema

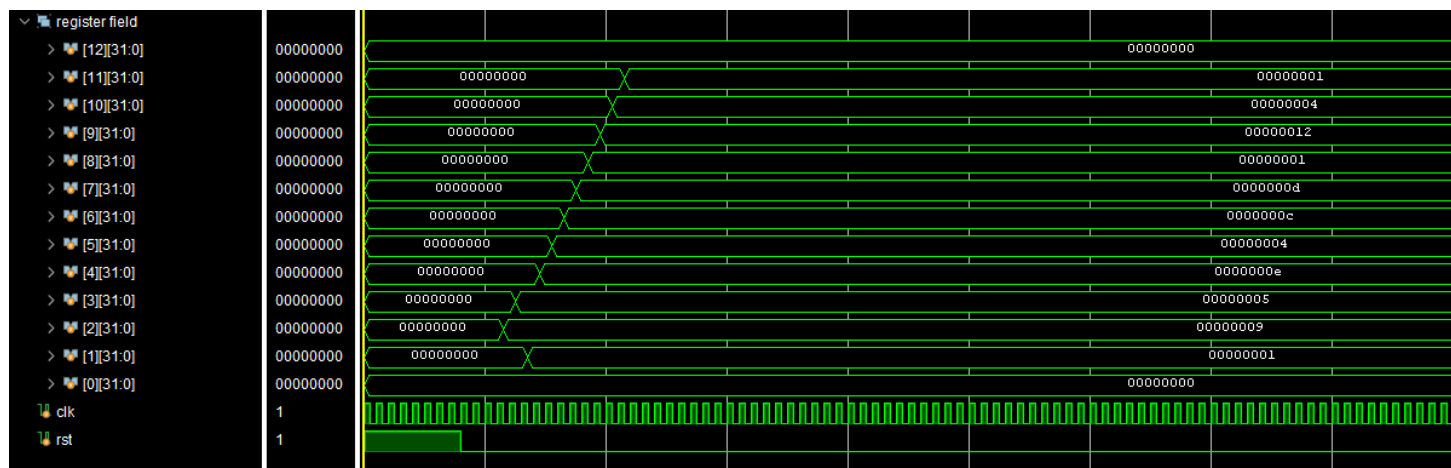
U nastavku biće prikazani asemblerski kodovi i talasni oblici kao dokaz ispravnosti sistema. Korišćen je Ripes simulator kojim je dati kod konvertovan u binaran zapis koji je zatim učitao u instrukcionu memoriju.

Ripes simulator poseduje i model pipeline procesora, taj model je korišćen kao referentni model sa čijim vrednostima iz registara su proveravane vrednosti u registrima implementiranog procesora.

Test implementiranih instrukcija R formata

```
addi x2, x0, 9
addi x3, x0, 5
addi x1, x0, 1
add x4, x2, x3
sub x5, x2, x3
xor x6, x2, x3
or x7, x2, x3
and x8, x2, x3
sll x9, x2, x1
srl x10, x2, x1
slt x11, x3, x2
```

Kodni segment iznad korišćen je za testiranje ispravnosti izvršavanja instrukcija R formata kod kojih su oba operanda smeštena u registre.



10. Talasni oblik izvršenih instrukcija R formata

Source code

Input type: ☒ Assembly ☐ C Executable code

View mode: ☐ Binary ☒ Disassembled

```
1 addi x2 x0 9
2 addi x3 x0 5
3 addi x1 x0 1
4 add x4 x2 x3
5 sub x5 x2 x3
6 xor x6 x2 x3
7 or x7 x2 x3
8 and x8 x2 x3
9 sll x9 x2 x1
10 srl x10 x2 x1
11 slt x11 x3 x2
```

```
0: 00900113 addi x2 x0 9
4: 00500193 addi x3 x0 5
8: 00100093 addi x1 x0 1
c: 00310233 add x4 x2 x3
10: 403102b3 sub x5 x2 x3
14: 00314333 xor x6 x2 x3
18: 003163b3 or x7 x2 x3
1c: 00317433 and x8 x2 x3
20: 001114b3 sll x9 x2 x1
24: 00115533 srl x10 x2 x1
28: 0021a5b3 slt x11 x3 x2
```

GPR

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000001
x2	sp	0x00000009
x3	gp	0x00000005
x4	tp	0x0000000e
x5	t0	0x00000004
x6	t1	0x0000000c
x7	t2	0x0000000d
x8	s0	0x00000001
x9	s1	0x00000012
x10	a0	0x00000004
x11	a1	0x00000001
x12	a2	0x00000000

11. Rezultati instrukcija unutar Ripes simulatora

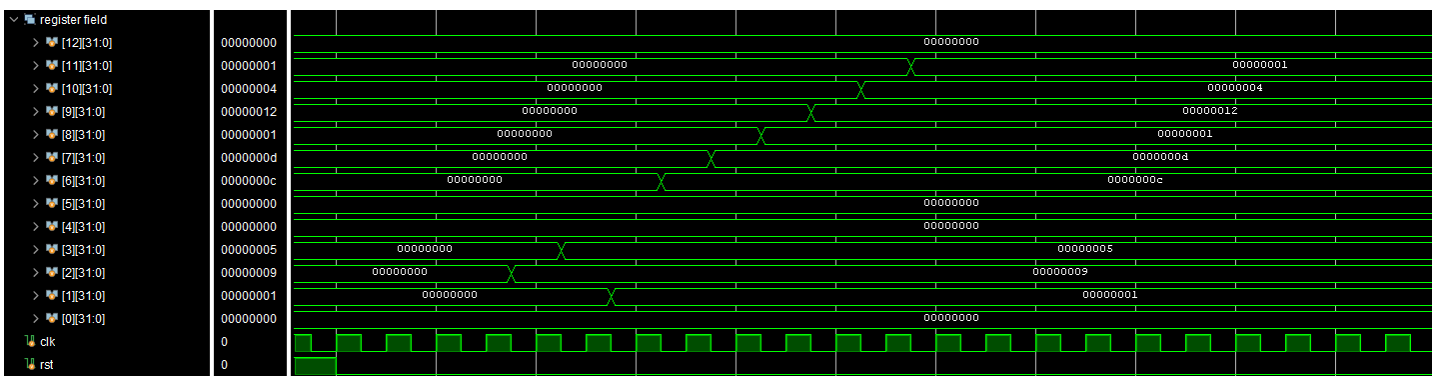
Test implementiranih instrukcija I formata

```

addi x2, x0, 9
addi x3, x0, 5
addi x1, x0, 1
xori x6, x2, 5
ori x7, x2, 5
andi x8, x2, 5
slli x9, x2, 1
srli x10, x2, 1
slti x11, x2, 10

```

Kodni segment iznad korišćen je za testiranje ispravnosti izvršavanja instrukcija I formata kod kojih je jedan operand smešten u registru, a drugi je upisan u instrukciju.



12. Talasni oblik izvršenih instrukcija I formata

Source code		Input type: ☒ Assembly ☐ C Executable code		View mode: ☐ Binary ☒ Disassembled		GPR		
1	addi x2 x0 9	0:	00900113	addi x2 x0 9		Name	Alias	Value
2	addi x3 x0 5	4:	00500193	addi x3 x0 5		x0	zero	0x00000000
3	addi x1 x0 1	8:	00100093	addi x1 x0 1		x1	ra	0x00000001
4	xori x6 x2 5	c:	00514313	xori x6 x2 5		x2	sp	0x00000009
5	ori x7 x2 5	10:	00516393	ori x7 x2 5		x3	gp	0x00000005
6	andi x8 x2 5	14:	00517413	andi x8 x2 5		x4	tp	0x00000000
7	slli x9 x2 1	18:	00111493	slli x9 x2 1		x5	t0	0x00000000
8	srli x10 x2 1	1c:	00115513	srli x10 x2 1		x6	t1	0x0000000c
9	slti x11 x2 10	20:	00a12593	slti x11 x2 10		x7	t2	0x0000000d
						x8	s0	0x00000001
						x9	s1	0x00000012
						x10	a0	0x00000004
						x11	a1	0x00000001

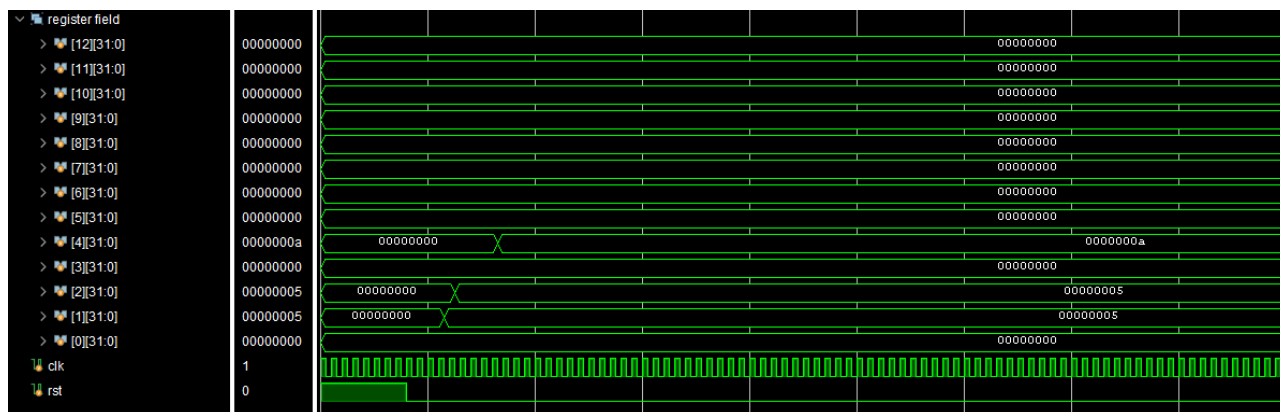
13. Rezultati instrukcija unutar Ripes simulatora

Test implementirane instrukcije B formata (uslovni skok)

addi x1, x0, 5	addi x1, x0, 5
addi x2, x0, 5	addi x2, x0, 4
beq x1, x2, equal	beq x1, x2, equal
addi x3, x0, 1	addi x3, x0, 1
equal:	equal:
addi x4, x0, 10	addi x4, x0, 10

Kodni segment iznad poseduje dva skupa instrukcija kod kojih je testirano da li je ispravno implementirana instrukcija uslovnog skoka u slučaju kada su vrednosti oba registra jednake.

U prvom slučaju su vrednosti u x1 i x2 iste, biće preskočena instrukcija “addi x3, x0, x1”, dok u drugom slučaju neće doći do skoka već će se obe instrukcije izvršiti ispod instrukcije beq.



14. Talasni oblik u slučaju izvršenog programskog skoka

U Ripes simulatoru inicijalna vrednost registra x3 je 0x10000000, zbog toga te dve vrednosti odstupaju. U registarskom polju u implementiranom procesoru inicijalna vrednost po resetu iznosi nula.

The screenshot shows the Ripes simulator interface. The 'Source code' panel on the left contains the following assembly code:

```

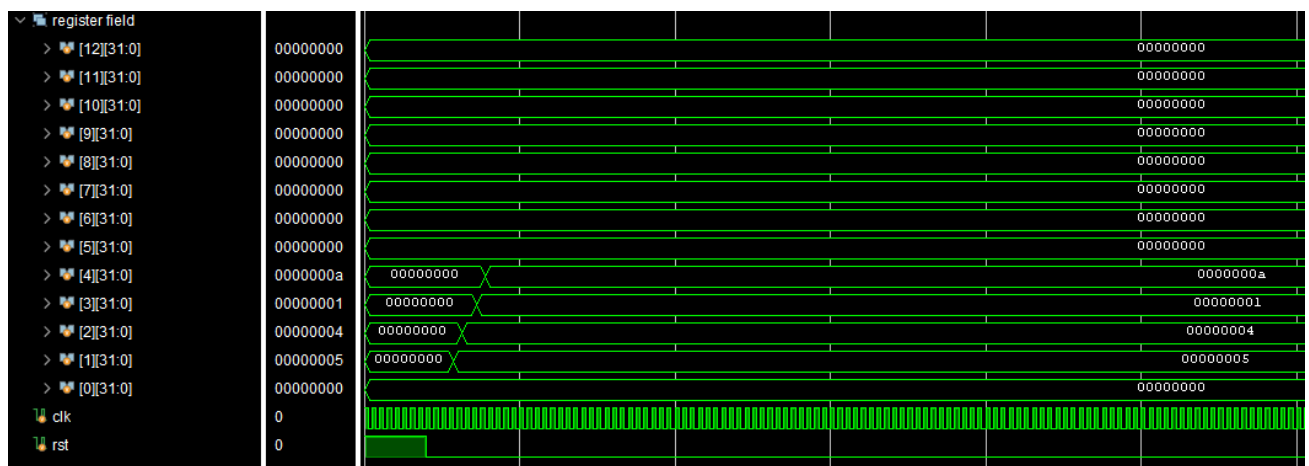
1 addi x1 x0 5
2 addi x2 x0 5
3 beq x1 x2 equal
4 addi x3 x0 1
5 equal:
6 addi x4 x0 10

```

The 'Input type' is set to 'Assembly'. The 'View mode' is set to 'Disassembled'. The assembly view shows the disassembled instructions with their addresses and values. The 'GPR' panel on the right shows the register values:

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000005
x2	sp	0x00000005
x3	gp	0x10000000
x4	tp	0x0000000a
x5	t0	0x00000000

15. Rezultati instrukcija unutar Ripes simulatora



16. Talasni oblik u slučaju ne izvršenog programskog skoka

The screenshot shows the Ripes simulator interface. The 'Source code' panel on the left contains the following assembly code:

```

1 addi x1 x0 5
2 addi x2 x0 4
3 beq x1 x2 equal
4 addi x3 x0 1
5 equal:
6 addi x4 x0 10

```

The 'Input type' is set to 'Assembly'. The 'View mode' is set to 'Disassembled'. The assembly view shows the disassembled instructions with their addresses and values. The 'GPR' panel on the right shows the register values:

Name	Alias	Value
x0	zero	0x00000000
x1	ra	0x00000005
x2	sp	0x00000004
x3	gp	0x00000001
x4	tp	0x0000000a
x5	t0	0x00000000

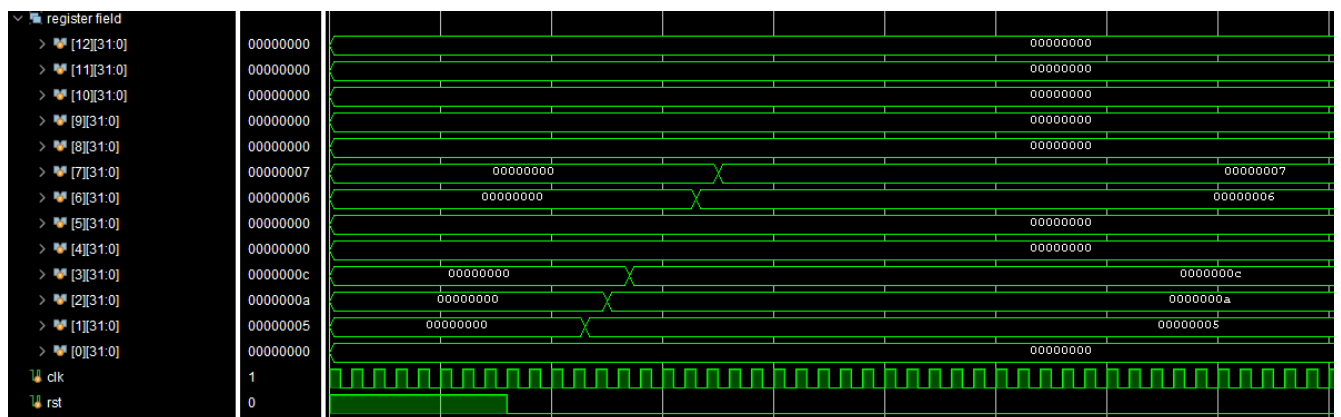
17. Rezultati instrukcija unutar Ripes simulatora

Test implementirane instrukcije J formata

```
addi x1, x0, 5
addi x2, x0, 10
jal x3, 12, func
addi x6, x0, 100
addi x7, x0, 200
```

func:

```
addi x6, x0, 6
addi x7, x0, 7
```



18. Talasni oblik izvršene instrukcije bezuslovnog skoka

Kodni segment iznad korišćen je za testiranje ispravnosti izvršavanja instrukcije bezuslovnog skoka (jal). U registar x6 upisana je vrednost 6, dok u registar x7 upisana je vrednost 7. Da funkcionalnost nije ispravno implementirana, ne bi došlo do skoka čime bi se nastavio program i prvo bi se u registar x6 upisala vrednost 100, a u registar x7 200, pa bi se onda tek upisala vrednost 6 i 7.

Source code		Input type: ☒ Assembly ☐ Executable code		View mode: ☐ Binary ☒ Disassembled		GPR																															
<pre>1 addi x1 x0 5 2 addi x2 x0 10 3 jal x3, func 4 addi x6 x0 100 5 addi x7 x0 200 6 7 func: 8 addi x6 x0 6 9 addi x7 x0 7</pre>		<pre>0: 00500093 addi x1 x0 5 4: 00a00113 addi x2 x0 10 8: 00c001ef jal x3 12 <func> c: 06400313 addi x6 x0 100 10: 0c800393 addi x7 x0 200 00000014 <func>: 14: 00600313 addi x6 x0 6 18: 00700393 addi x7 x0 7</pre>				<table><thead><tr><th>Name</th><th>Alias</th><th>Value</th></tr></thead><tbody><tr><td>x0</td><td>zero</td><td>0x00000000</td></tr><tr><td>x1</td><td>ra</td><td>0x00000005</td></tr><tr><td>x2</td><td>sp</td><td>0x0000000a</td></tr><tr><td>x3</td><td>gp</td><td>0x0000000c</td></tr><tr><td>x4</td><td>tp</td><td>0x00000000</td></tr><tr><td>x5</td><td>t0</td><td>0x00000000</td></tr><tr><td>x6</td><td>t1</td><td>0x00000006</td></tr><tr><td>x7</td><td>t2</td><td>0x00000007</td></tr><tr><td>x8</td><td>s0</td><td>0x00000000</td></tr></tbody></table>		Name	Alias	Value	x0	zero	0x00000000	x1	ra	0x00000005	x2	sp	0x0000000a	x3	gp	0x0000000c	x4	tp	0x00000000	x5	t0	0x00000000	x6	t1	0x00000006	x7	t2	0x00000007	x8	s0	0x00000000
Name	Alias	Value																																			
x0	zero	0x00000000																																			
x1	ra	0x00000005																																			
x2	sp	0x0000000a																																			
x3	gp	0x0000000c																																			
x4	tp	0x00000000																																			
x5	t0	0x00000000																																			
x6	t1	0x00000006																																			
x7	t2	0x00000007																																			
x8	s0	0x00000000																																			

19. Rezultati instrukcija unutar Ripes simulatora

Test evikcije i operacije čitanja u registar

addi x1, x0, 256	addi x1, x0, 256	addi x1, x0, 256
addi x2, x0, 512	addi x2, x0, 512	addi x2, x0, 512
addi x3, x0, 1024	addi x3, x0, 1024	addi x3, x0, 1024
add x4, x3, x3	add x4, x3, x3	add x4, x3, x3
add x5, x4, x4	add x5, x4, x4	add x5, x4, x4
lui x6, 0x2	lui x6, 0x2	lui x6, 0x2
lui x7, 0x4	lui x7, 0x4	lui x7, 0x4
lui x8, 0x8	lui x8, 0x8	lui x8, 0x8
sw x1, 0(x0)	sw x1, 12(x0)	sw x1, 20(x0)
sw x2, 0(x1)	sw x2, 12(x1)	sw x2, 20(x1)
sw x3, 0(x2)	sw x3, 12(x2)	sw x3, 20(x2)
sw x4, 0(x3)	sw x4, 12(x3)	sw x4, 20(x3)
sw x5, 0(x4)	sw x5, 12(x4)	sw x5, 20(x4)
sw x6, 0(x5)	sw x6, 12(x5)	sw x6, 20(x5)
sw x7, 0(x6)	sw x7, 12(x6)	sw x7, 20(x6)
lw x2, 0(x0)	lw x2, 12(x0)	lw x2, 20(x0)
addi x0, x0, 0	addi x0, x0, 0	addi x0, x0, 0

Cilj ovog kodnog segmenta je da se izazove evikcija iz L1 i L2 keš memorija i da se prvi eviktovani podatak učita iz glavne memorije nazad u registarsko polje. Data su tri kodna segmenta, u svakoj koloni je ista logika, samo se u različite setove u keš memoriji upisuju podaci. Obe keš memorije su bajt adresibilne, prva adresa je nula. U prvoj koloni podaci se upisuju u nulti set, u drugoj koloni podaci se upisuju u treći set ($12/4 = 3$) i u trećoj koloni podaci se upisuju u peti set ($20/4 = 5$).

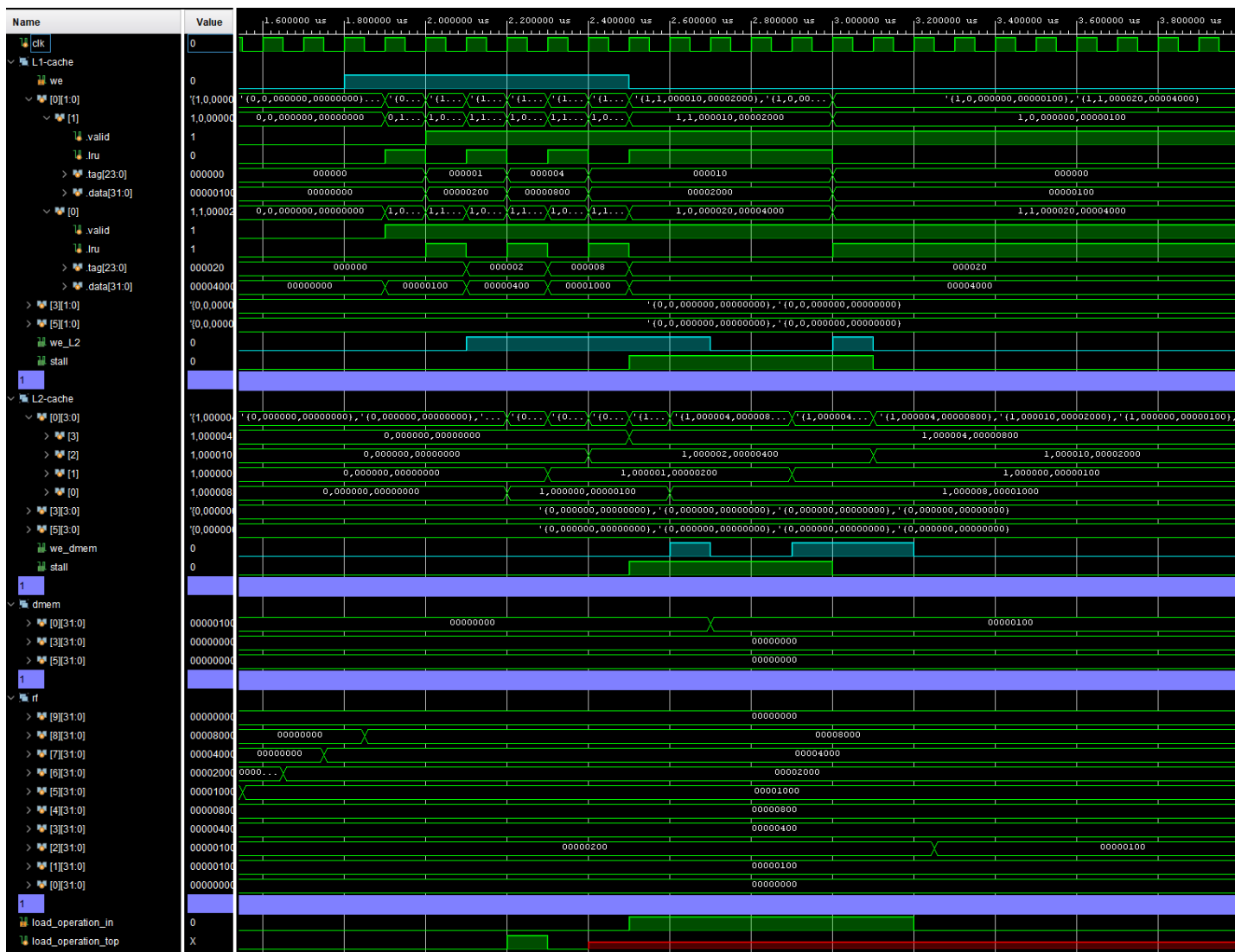
Na početku se u registre smeštaju vrednosti koje će se koristiti kako bi moglo u te specifične adrese da se upiše podatak. Cilj je da se svi smerovi obe keš memorije popune sa podacima čiji su tagovi različiti.

Redosled upisa podataka : prvo se u nulti smer nultog seta L1 keš memorije upisuje podatak 256 iz registra x1 sa tagom nula, zatim se u prvi smer nultog

seta upisuje podatak iz registra x2 sa tagom 0x1. Sledeći upis u L1 izaziva evikciju podatka iz nultog smera iz L1 u nulti smer L2. Ponovlja se scenario sve dok se ne popune sva 4 smera i u L2 memoriji. Nakon toga ponovo se poziva operacija upisa u memoriju čime se podatak sa nultog smera iz nultog seta L2 memorije eviktuje u glavnu memoriju. Nakon toga se poziva operacija čitanja iz memorije. S obzirom da traženi podatak nije prisutan ni u L1 keš memoriji ni u L2 keš memoriji potrebno je učitati taj podatak prvo u L2, pa se onda taj podatak šalje i ka L1 da bi se na kraju upisao i u registar x2.

Može se videti u sva tri slučaja da u slučaju promašaja pri load instrukciji u L1 i u L2 stall signal postaje jedinica i po učitavanju podatka iz glavne memorije u L2 dolazi do cache hit-a i interni stall L2 keš memorije postaje nula. U sledećem taktu taj podatak se učitava i u L1 keš memoriju nakon čega i u L1 keš memoriji dolazi do cache hit-a nakon čega i taj stall postaje nula.

U nastavku biće prikazani talasni oblici koji odgovaraju asemblerskim kodovima prikazani u kodnom listingu.



20. Evikcija podatka iz seta nula u glavnu memoriju i čitanje podatka iz glavne memorije

Source code	Input type: ☒ Assembly ☐ C Executable code	View mode: ☐ Binary ☒ Disassembled	GPR
<pre>1 addi x1, x0, 0x100 2 addi x2, x0, 0x200 3 addi x3, x0, 0x400 4 add x4, x3, x3 5 add x5, x4, x4 6 lui x6, 2 7 lui x7, 4 8 lui x8, 8 9 10 sw x1, 0(x0) 11 sw x2, 0(x1) 12 sw x3, 0(x2) 13 sw x4, 0(x3) 14 sw x5, 0(x4) 15 sw x6, 0(x5) 16 sw x7, 0(x6) 17 lw x2, 0(x0) 18 nop</pre>	</		

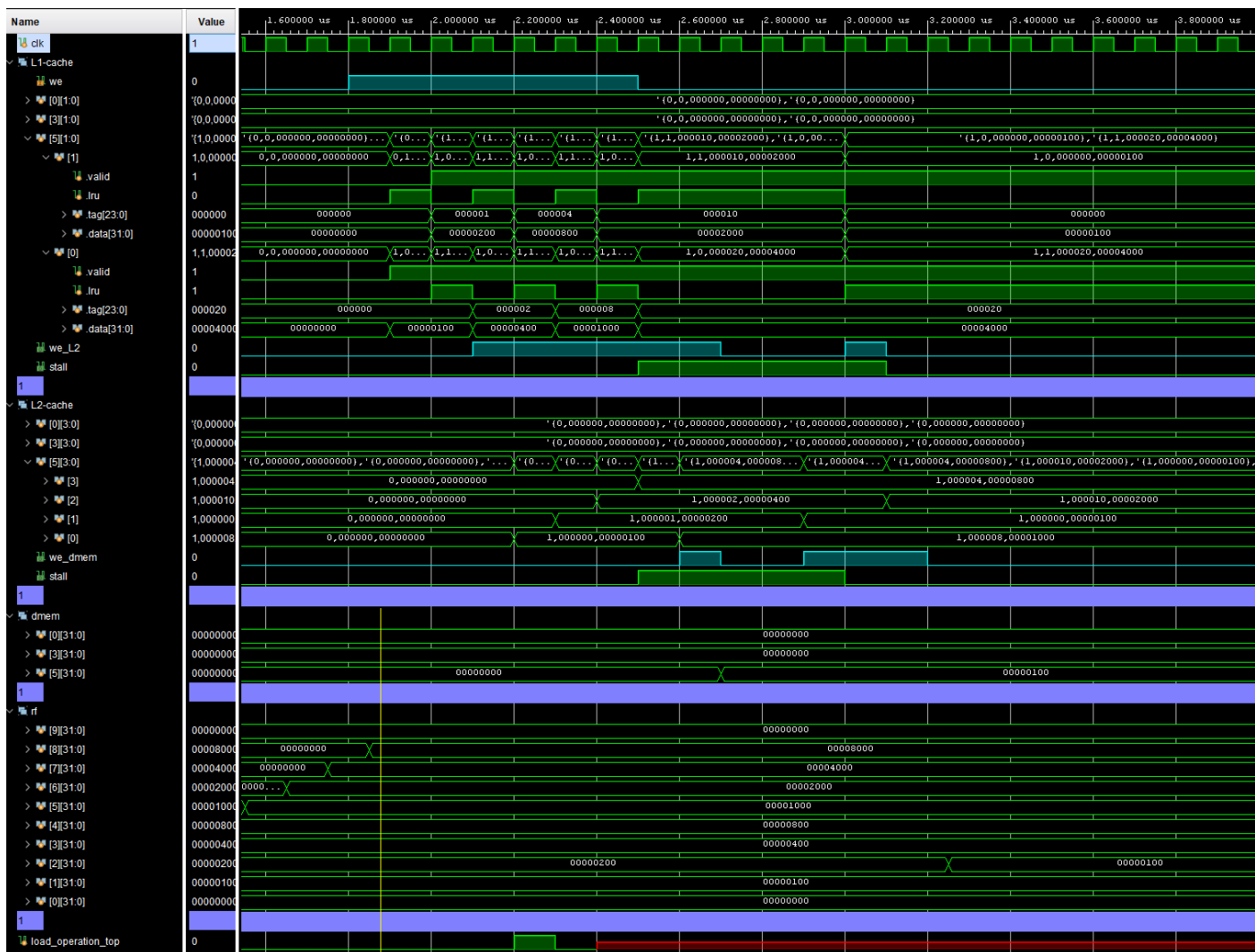
21. Rezultati Ripes simulatora za kodni segment prve kolone



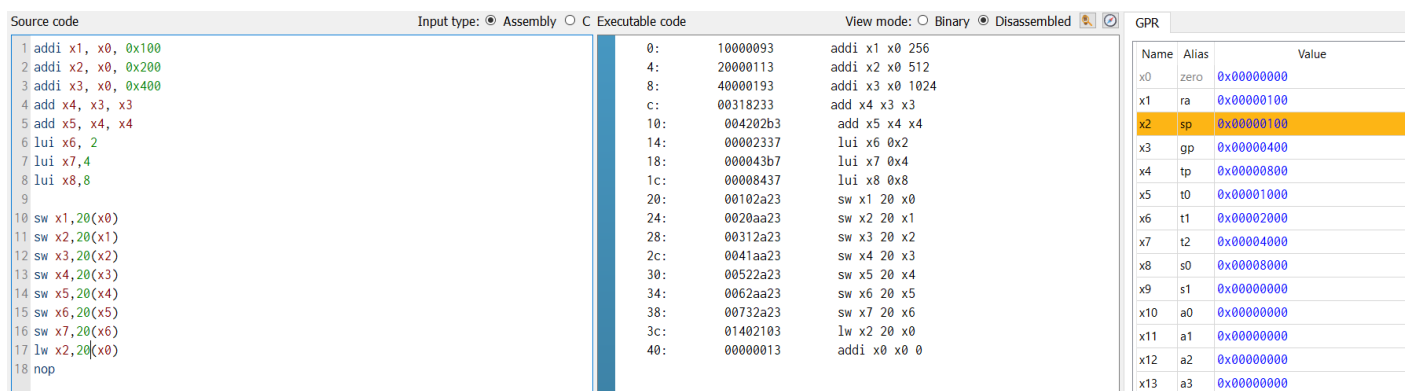
22. Evikcija podataka iz seta tri u glavnu memoriju i čitanje podatka iz glavne memorije

Source code	Input type: ☒ Assembly ☐ C Executable code	View mode: ☐ Binary ☒ Disassembled	GPR																																												
1 addi x1, x0, 0x100 2 addi x2, x0, 0x200 3 addi x3, x0, 0x400 4 add x4, x3, x3 5 add x5, x4, x4 6 lui x6, 2 7 lui x7, 4 8 lui x8, 8 9 10 sw x1, 12(x0) 11 sw x2, 12(x1) 12 sw x3, 12(x2) 13 sw x4, 12(x3) 14 sw x5, 12(x4) 15 sw x6, 12(x5) 16 sw x7, 12(x6) 17 lw x2, 12(x0) 18 nop	0: 10000093 addi x1 x0 256 4: 20000113 addi x2 x0 512 8: 40000193 addi x3 x0 1024 c: 00318233 add x4 x3 x3 10: 004202b3 add x5 x4 x4 14: 00002337 lui x6 0x2 18: 000043b7 lui x7 0x4 1c: 00008437 lui x8 0x8 20: 00102623 sw x1 12 x0 24: 0020a623 sw x2 12 x1 28: 00312623 sw x3 12 x2 2c: 0041a623 sw x4 12 x3 30: 00522623 sw x5 12 x4 34: 0062a623 sw x6 12 x5 38: 00732623 sw x7 12 x6 3c: 00c02103 lw x2 12 x0 40: 00000013 addi x0 x0 0	<table><thead><tr><th>Name</th><th>Alias</th><th>Value</th></tr></thead><tbody><tr><td>x0</td><td>zero</td><td>0x00000000</td></tr><tr><td>x1</td><td>ra</td><td>0x00000100</td></tr><tr><td>x2</td><td>sp</td><td>0x00000100</td></tr><tr><td>x3</td><td>gp</td><td>0x00000400</td></tr><tr><td>x4</td><td>tp</td><td>0x00000800</td></tr><tr><td>x5</td><td>t0</td><td>0x00001000</td></tr><tr><td>x6</td><td>t1</td><td>0x00002000</td></tr><tr><td>x7</td><td>t2</td><td>0x00004000</td></tr><tr><td>x8</td><td>s0</td><td>0x00008000</td></tr><tr><td>x9</td><td>s1</td><td>0x00000000</td></tr><tr><td>x10</td><td>a0</td><td>0x00000000</td></tr><tr><td>x11</td><td>a1</td><td>0x00000000</td></tr><tr><td>x12</td><td>a2</td><td>0x00000000</td></tr><tr><td>x13</td><td>a3</td><td>0x00000000</td></tr></tbody></table>	Name	Alias	Value	x0	zero	0x00000000	x1	ra	0x00000100	x2	sp	0x00000100	x3	gp	0x00000400	x4	tp	0x00000800	x5	t0	0x00001000	x6	t1	0x00002000	x7	t2	0x00004000	x8	s0	0x00008000	x9	s1	0x00000000	x10	a0	0x00000000	x11	a1	0x00000000	x12	a2	0x00000000	x13	a3	0x00000000
Name	Alias	Value																																													
x0	zero	0x00000000																																													
x1	ra	0x00000100																																													
x2	sp	0x00000100																																													
x3	gp	0x00000400																																													
x4	tp	0x00000800																																													
x5	t0	0x00001000																																													
x6	t1	0x00002000																																													
x7	t2	0x00004000																																													
x8	s0	0x00008000																																													
x9	s1	0x00000000																																													
x10	a0	0x00000000																																													
x11	a1	0x00000000																																													
x12	a2	0x00000000																																													
x13	a3	0x00000000																																													

23. Rezultati Ripes simulatora za kodni segment druge kolone



24. Evikcija podataka iz seta pet u glavnu memoriju i čitanje podatka iz glavne memorije



25. Rezultati Ripes simulatora za kodni segment treće kolone

Analiza utrošenosti resursa

Tip modula	Iskorišćeno	Ukupan broj resursa	Utrošenost
LUT	31668	78600	40.09%
FF	89703	157200	57.06%

26. Utrošenost hardverskih resursa

Timing	Setup	Hold	Pulse Width
Worst Pulse Width Slack (WPWS):		99.6 ns	
Total Pulse Width Negative Slack (TPWS):		0 ns	
Number of Failing Endpoints:		0	
Total Number of Endpoints:		89703	

27. Procena frekvencije rada

```
create_clock -period 200 -name clk -waveform {0.000 100.000} [get_ports clk]
```

Pokretanjem implementacije uz constraint naveden u kodnom segmentu iznad frekvencija rada iznosi 9.96MHz.

Frekvencija se računa pomoću formule $\frac{1}{T-WNS}$.